Tối ưu hiệu năng PostgreSQL

Tài liệu đào tạo nội bộ cho kỹ sư backend và quản trị cơ sở dữ liệu

Phiên bản 1.2 – Tài liệu mẫu dùng cho dự án AI Learning Assistant

Tài liệu này tổng hợp các kỹ thuật thực tế để chẩn đoán và cải thiện hiệu năng của PostgreSQL trong các hệ thống web có tải trung bình đến cao. Nội dung tập trung vào những việc mang lại hiệu quả lớn nhất với chi phí thấp nhất: đọc kế hoạch thực thi, thiết kế chỉ mục đúng, cấu hình bộ nhớ hợp lý, kiểm soát số lượng kết nối, bảo trì bằng VACUUM, phân vùng bảng và giám sát liên tục.

Tài liệu giả định người đọc đã quen với SQL cơ bản và biết dùng công cụ dòng lệnh psql. Các con số trong phần ví dụ và tình huống thực tế là số liệu minh họa, không phải kết quả đo từ một hệ thống cụ thể.

Mục lục

1. Cách PostgreSQL thực thi một truy vấn
2. Chỉ mục (index)
3. Những lỗi truy vấn thường gặp
4. Cấu hình bộ nhớ và kết nối
5. MVCC, VACUUM và hiện tượng phình bảng
6. Phân vùng bảng (partitioning)
7. Replication, sao lưu và tính sẵn sàng cao
8. Giám sát, tình huống thực tế và danh sách kiểm tra

1. Cách PostgreSQL thực thi một truy vấn

Khi nhận một câu lệnh SQL, PostgreSQL đi qua bốn giai đoạn: phân tích cú pháp (parser), viết lại (rewriter), lập kế hoạch (planner) và thực thi (executor). Giai đoạn quan trọng nhất đối với hiệu năng là lập kế hoạch. Planner dựa trên thống kê của bảng, do lệnh ANALYZE thu thập, để ước tính chi phí của nhiều kế hoạch khác nhau rồi chọn kế hoạch có chi phí ước tính thấp nhất.

Chi phí được tính theo đơn vị tùy ý chứ không phải mili giây. Hai tham số quan trọng là seq_page_cost (mặc định 1.0) và random_page_cost (mặc định 4.0). Giá trị 4.0 phản ánh ổ đĩa quay; với ổ SSD nên giảm random_page_cost xuống khoảng 1.1 để planner sẵn sàng chọn chỉ mục hơn.

Lệnh EXPLAIN chỉ hiển thị kế hoạch dự kiến, còn EXPLAIN (ANALYZE, BUFFERS) thực sự chạy truy vấn và báo thời gian thực tế, số dòng thực tế và số block đọc từ bộ nhớ đệm hay từ đĩa. Lưu ý rằng EXPLAIN ANALYZE thực thi câu lệnh thật, vì vậy với INSERT, UPDATE hoặc DELETE phải đặt trong một transaction rồi ROLLBACK để không làm thay đổi dữ liệu.

Có bốn kiểu quét bảng chính. Seq Scan đọc toàn bộ bảng. Index Scan dùng chỉ mục để tìm dòng rồi đọc bảng. Index Only Scan chỉ đọc chỉ mục, với điều kiện mọi cột cần thiết đã nằm trong chỉ mục và trang dữ liệu được đánh dấu là nhìn thấy được trong visibility map. Bitmap Heap Scan gom các vị trí từ một hoặc nhiều chỉ mục rồi đọc bảng theo thứ tự vật lý.

Với phép JOIN, planner chọn giữa ba thuật toán. Nested Loop tốt khi một bên rất nhỏ và bên kia có chỉ mục. Hash Join thường tốt nhất cho các tập dữ liệu lớn chưa được sắp xếp. Merge Join phù hợp khi hai bên đã được sắp xếp theo khóa nối.

Seq Scan không phải lúc nào cũng xấu: với bảng nhỏ, hoặc khi truy vấn cần đọc phần lớn bảng, quét tuần tự nhanh hơn việc nhảy qua lại giữa chỉ mục và bảng.

2. Chỉ mục (index)

Loại chỉ mục mặc định trong PostgreSQL là B-tree, phù hợp cho các phép so sánh =, <, >, BETWEEN và cho ORDER BY. Ngoài ra còn có Hash (chỉ hỗ trợ so sánh bằng), GIN (tối ưu cho mảng, jsonb và tìm kiếm toàn văn), GiST (dữ liệu hình học và khoảng giá trị), SP-GiST và BRIN. BRIN chỉ lưu giá trị nhỏ nhất và lớn nhất của từng nhóm trang liên tiếp, nên rất nhỏ gọn và phù hợp với bảng cực lớn mà dữ liệu được ghi gần như theo thứ tự thời gian, chẳng hạn bảng log.

Chỉ mục nhiều cột (composite index) tuân theo quy tắc tiền tố bên trái. Chỉ mục (customer_id, created_at) phục vụ tốt cho truy vấn lọc theo customer_id, hoặc theo cả customer_id và created_at, nhưng gần như không giúp ích cho truy vấn chỉ lọc theo created_at. Nguyên tắc thường dùng là đặt cột lọc bằng (=) trước, rồi đến cột lọc theo khoảng hoặc cột sắp xếp.

Chỉ mục một phần (partial index) chỉ lập chỉ mục cho các dòng thỏa một điều kiện, ví dụ CREATE INDEX ON orders (created_at) WHERE status = 'pending'. Chỉ mục này nhỏ hơn nhiều và nhanh hơn khi hệ thống thường xuyên truy vấn các đơn chưa xử lý, vốn chỉ chiếm một phần nhỏ của bảng.

Chỉ mục bao phủ (covering index) dùng mệnh đề INCLUDE, có từ PostgreSQL 11, để thêm vào chỉ mục những cột không dùng làm khóa tìm kiếm. Nhờ đó truy vấn đạt được Index Only Scan mà không phải đọc bảng.

Chỉ mục không miễn phí. Mỗi chỉ mục làm chậm INSERT, UPDATE và DELETE vì phải cập nhật thêm, đồng thời chiếm dung lượng đĩa và bộ nhớ đệm. Quy tắc thực hành là chỉ tạo chỉ mục khi có truy vấn thật sự cần, và định kỳ xóa các chỉ mục không dùng bằng cách kiểm tra cột idx_scan trong view pg_stat_user_indexes.

Khi tạo chỉ mục trên bảng đang chạy production, hãy dùng CREATE INDEX CONCURRENTLY để không chặn các thao tác ghi. Đổi lại, thời gian tạo chỉ mục sẽ lâu hơn và lệnh không thể chạy bên trong một transaction.

3. Những lỗi truy vấn thường gặp

• Dùng hàm trên cột được lọc. Điều kiện WHERE lower(email) = 'a@example.com' không dùng được chỉ mục thường trên cột email. Giải pháp là tạo chỉ mục biểu thức: CREATE INDEX ON users (lower(email)). Tương tự, so sánh hai cột khác kiểu dữ liệu có thể làm PostgreSQL ép kiểu ngầm và bỏ qua chỉ mục.

• SELECT * khi chỉ cần vài cột. Cách viết này kéo về nhiều dữ liệu không cần thiết, tăng băng thông mạng và ngăn truy vấn đạt Index Only Scan. Hãy liệt kê rõ các cột cần dùng.

• OFFSET lớn. Với OFFSET 100000, PostgreSQL vẫn phải đọc rồi bỏ đi 100000 dòng đầu. Thay vào đó hãy dùng phân trang theo khóa (keyset pagination): WHERE (created_at, id) < (:last_created_at, :last_id) ORDER BY created_at DESC, id DESC LIMIT 20. Thời gian truy vấn gần như không đổi dù người dùng đi sâu đến trang nào.

• Vấn đề N+1. ORM chạy một truy vấn lấy danh sách rồi chạy thêm N truy vấn để lấy chi tiết cho từng dòng. Với 500 dòng sẽ có 501 lần đi về giữa ứng dụng và cơ sở dữ liệu. Hãy dùng JOIN, nạp trước dữ liệu liên quan (eager loading) hoặc gom lại bằng WHERE id = ANY(:ids).

• Tìm kiếm LIKE '%từ khóa%'. Mẫu bắt đầu bằng ký tự đại diện không dùng được chỉ mục B-tree. Hãy dùng extension pg_trgm với chỉ mục GIN, hoặc tìm kiếm toàn văn nếu bài toán phù hợp.

• Thống kê lỗi thời. Nếu trong EXPLAIN ANALYZE số dòng ước tính (rows) lệch với số dòng thực tế (actual rows) hàng chục lần, planner đang làm việc với thống kê cũ hoặc sai. Hãy chạy ANALYZE trên bảng đó và kiểm tra lại.

• Transaction mở quá lâu. Một transaction ở trạng thái 'idle in transaction' vẫn giữ khóa và ngăn VACUUM dọn các dòng chết, dẫn đến phình bảng. Nên đặt idle_in_transaction_session_timeout để tự ngắt những phiên như vậy.

4. Cấu hình bộ nhớ và kết nối

shared_buffers là vùng bộ nhớ đệm dùng chung của PostgreSQL. Giá trị mặc định chỉ là 128MB; điểm khởi đầu phổ biến cho máy chuyên dụng là khoảng 25% dung lượng RAM. Tăng quá cao thường không có lợi vì hệ điều hành cũng đã có bộ nhớ đệm tệp riêng.

effective_cache_size không cấp phát bộ nhớ. Nó chỉ là gợi ý cho planner về tổng lượng bộ nhớ đệm khả dụng, gồm cả bộ nhớ đệm của hệ điều hành, và thường được đặt trong khoảng 50% đến 75% RAM.

work_mem (mặc định 4MB) là bộ nhớ cho mỗi thao tác sắp xếp hoặc băm bên trong một truy vấn. Một truy vấn phức tạp có thể dùng work_mem nhiều lần, và mỗi kết nối đều có thể làm như vậy, nên tổng bộ nhớ tiềm năng xấp xỉ max_connections nhân work_mem nhân số thao tác. Vì vậy không nên đặt giá trị này quá cao trên toàn hệ thống. Dấu hiệu thiếu work_mem là dòng 'Sort Method: external merge Disk' trong EXPLAIN ANALYZE, nghĩa là việc sắp xếp phải tràn ra đĩa.

maintenance_work_mem (mặc định 64MB) dùng cho các tác vụ bảo trì như VACUUM và CREATE INDEX. Trên máy lớn có thể đặt từ 512MB đến 2GB để các tác vụ này chạy nhanh hơn.

max_connections mặc định là 100. Mỗi kết nối PostgreSQL là một tiến trình riêng của hệ điều hành, tốn bộ nhớ và chi phí chuyển ngữ cảnh. Nhiều kết nối không đồng nghĩa với nhiều thông lượng: khi số kết nối hoạt động vượt xa số lõi CPU, hiệu năng thường giảm. Một công thức khởi đầu hay được trích dẫn cho kích thước pool là số lõi CPU nhân 2 cộng số ổ đĩa hiệu dụng; đây chỉ là điểm xuất phát cần đo lại.

Giải pháp là dùng bộ gom kết nối (connection pooler) như PgBouncer. PgBouncer có ba chế độ: session pooling, transaction pooling và statement pooling. Transaction pooling được dùng phổ biến nhất vì cho phép hàng nghìn client dùng chung vài chục kết nối thật. Tuy nhiên chế độ này không tương thích với các tính năng gắn với phiên làm việc như advisory lock mức phiên hay lệnh SET mức phiên.

Ví dụ minh họa cho máy chủ 16GB RAM, 8 lõi CPU: shared_buffers = 4GB, effective_cache_size = 12GB, work_mem = 16MB, maintenance_work_mem = 1GB, max_connections = 200, và PgBouncer ở chế độ transaction pooling với kích thước pool từ 20 đến 30 kết nối.

5. MVCC, VACUUM và hiện tượng phình bảng

PostgreSQL dùng MVCC (Multi-Version Concurrency Control): thao tác đọc không chặn thao tác ghi và ngược lại. Cái giá phải trả là mỗi lệnh UPDATE không sửa dòng tại chỗ mà tạo ra một phiên bản dòng mới, còn phiên bản cũ trở thành dòng chết (dead tuple). Nếu dòng chết không được dọn, bảng và chỉ mục sẽ phình to (bloat) và truy vấn chậm dần.

VACUUM thu hồi chỗ của các dòng chết để tái sử dụng nhưng thường không trả dung lượng về cho hệ điều hành. VACUUM FULL viết lại toàn bộ bảng và trả dung lượng, nhưng cần khóa độc quyền ACCESS EXCLUSIVE trong suốt thời gian chạy nên cần tránh trên production. Có thể dùng extension pg_repack để đạt hiệu quả tương tự mà gần như không chặn truy cập.

Autovacuum tự động chạy VACUUM trên một bảng khi số dòng chết vượt ngưỡng: autovacuum_vacuum_threshold (mặc định 50) cộng với autovacuum_vacuum_scale_factor (mặc định 0.2) nhân với số dòng của bảng. Ví dụ, bảng 10 triệu dòng chỉ được dọn khi có khoảng 2.000.050 dòng chết, tức là rất muộn. Với bảng lớn nên giảm scale factor riêng cho từng bảng, chẳng hạn ALTER TABLE orders SET (autovacuum_vacuum_scale_factor = 0.01), khi đó ngưỡng chỉ còn khoảng 100.050 dòng chết. Tham số autovacuum_analyze_scale_factor mặc định là 0.1 và điều khiển việc cập nhật thống kê tương tự.

HOT update (Heap-Only Tuple) xảy ra khi lệnh UPDATE không thay đổi cột nào có chỉ mục và trang dữ liệu còn chỗ trống, nhờ đó PostgreSQL không cần tạo thêm mục trong chỉ mục. Có thể chừa sẵn chỗ trống bằng cách đặt fillfactor cho bảng, ví dụ 90 hoặc 80, đổi lại bảng chiếm nhiều dung lượng hơn.

Mã giao dịch (transaction ID) của PostgreSQL là số 32 bit nên có nguy cơ quay vòng (wraparound). VACUUM định kỳ 'đóng băng' (freeze) các dòng cũ để tránh điều này. Khi tuổi của bảng chạm autovacuum_freeze_max_age (mặc định 200 triệu), autovacuum sẽ chạy cưỡng bức ngay cả khi đã bị tắt. Transaction chạy quá lâu hoặc replication slot bị bỏ quên có thể ngăn việc dọn dẹp này.

Để theo dõi, hãy xem cột n_dead_tup và last_autovacuum trong view pg_stat_user_tables. Bảng có tỷ lệ dòng chết cao mà last_autovacuum đã rất cũ là dấu hiệu autovacuum đang chạy không đủ tích cực.

6. Phân vùng bảng (partitioning)

Từ PostgreSQL 10, phân vùng khai báo (declarative partitioning) hỗ trợ ba kiểu: phân vùng theo khoảng (range), theo danh sách (list) và theo băm (hash). Bảng cha không chứa dữ liệu; dữ liệu nằm trong các bảng con gọi là partition.

Lợi ích thứ nhất là partition pruning: khi truy vấn có điều kiện trên khóa phân vùng, planner chỉ quét những partition liên quan và bỏ qua phần còn lại. Lợi ích thứ hai là bảo trì dữ liệu cũ: xóa một partition bằng DROP TABLE hoặc ALTER TABLE ... DETACH PARTITION nhanh hơn rất nhiều so với DELETE hàng triệu dòng, và không tạo ra dòng chết cần VACUUM. Ngoài ra, mỗi partition có chỉ mục và thống kê nhỏ hơn nên VACUUM và ANALYZE cũng nhẹ hơn.

Không nên phân vùng mọi bảng. Phân vùng phù hợp với bảng đạt vài chục GB trở lên hoặc hàng trăm triệu dòng, và phần lớn truy vấn đều có kèm khóa phân vùng, thường là cột thời gian như created_at. Với bảng nhỏ, phân vùng chỉ làm tăng chi phí lập kế hoạch mà không mang lại lợi ích.

Có một ràng buộc dễ gây bất ngờ: khóa chính và mọi ràng buộc UNIQUE trên bảng phân vùng bắt buộc phải bao gồm các cột của khóa phân vùng. Vì vậy khóa chính (id) cần đổi thành (id, created_at) nếu phân vùng theo created_at. Cũng nên giữ số lượng partition ở mức vài chục đến vài trăm, vì hàng nghìn partition làm thời gian lập kế hoạch tăng đáng kể.

Ví dụ minh họa: bảng events được phân vùng theo tháng với các partition events_2026_01, events_2026_02 và tiếp tục như vậy. Hệ thống giữ dữ liệu 13 tháng gần nhất. Mỗi tháng, một tác vụ định kỳ tạo trước partition cho tháng kế tiếp và tách (detach) rồi lưu trữ partition cũ nhất. Nên tạo thêm một partition DEFAULT để hứng các dòng không khớp phạm vi nào, tránh lỗi khi chèn dữ liệu bất thường.

7. Replication, sao lưu và tính sẵn sàng cao

Streaming replication (nhân bản vật lý) chuyển nhật ký WAL từ máy chủ chính sang các máy replica. Replica có thể phục vụ truy vấn chỉ đọc như báo cáo, giúp giảm tải cho máy chủ chính. Mặc định việc nhân bản là bất đồng bộ, nên tồn tại độ trễ nhân bản (replication lag): một ứng dụng đọc ngay sau khi ghi có thể thấy dữ liệu cũ nếu đọc từ replica. Thao tác nhạy cảm với tính nhất quán nên đọc từ máy chủ chính.

Có thể chọn nhân bản đồng bộ bằng synchronous_commit và synchronous_standby_names. Khi đó giao dịch chỉ được xác nhận sau khi replica đã nhận WAL, nên không mất dữ liệu khi chuyển đổi dự phòng, đổi lại độ trễ của các thao tác ghi tăng lên. Với logical replication, ta nhân bản theo từng bảng và có thể giữa các phiên bản PostgreSQL khác nhau, rất hữu ích khi nâng cấp phiên bản với thời gian ngừng dịch vụ tối thiểu.

Truy vấn dài trên replica có thể bị hủy vì xung đột với quá trình áp dụng WAL (lỗi 'conflict with recovery'). Có thể giảm vấn đề này bằng hot_standby_feedback hoặc tăng max_standby_streaming_delay, nhưng cần hiểu rằng hot_standby_feedback có thể làm máy chủ chính giữ lại dòng chết lâu hơn.

Việc tự động chuyển đổi dự phòng (failover) cần công cụ chuyên dụng như Patroni hoặc repmgr, cùng cơ chế chống split-brain, tức là hai máy cùng tin mình là máy chủ chính. Replication slot giúp máy chủ chính giữ lại WAL cho replica chưa kịp nhận, nhưng một slot bị bỏ quên sẽ khiến WAL tích tụ và có thể làm đầy ổ đĩa.

Về sao lưu có hai nhóm. pg_dump tạo bản sao lưu logic, tiện cho việc chuyển dữ liệu hoặc khôi phục từng bảng nhưng chậm với cơ sở dữ liệu lớn. pg_basebackup kết hợp lưu trữ WAL liên tục (WAL archiving) cho phép khôi phục về một thời điểm bất kỳ (PITR, point-in-time recovery). Khi đặt mục tiêu RPO, tức lượng dữ liệu có thể chấp nhận mất, bằng 5 phút, cần có WAL archiving liên tục. Một nguyên tắc quan trọng: bản sao lưu chưa từng được thử khôi phục thì chưa thể coi là bản sao lưu.

8. Giám sát, tình huống thực tế và danh sách kiểm tra

Extension pg_stat_statements là công cụ giám sát quan trọng nhất. Cần thêm nó vào shared_preload_libraries và khởi động lại máy chủ. Từ PostgreSQL 13, các cột chính là calls, total_exec_time và mean_exec_time (các phiên bản trước dùng total_time và mean_time). Hãy sắp xếp theo total_exec_time để tìm các truy vấn tiêu tốn nhiều tài nguyên nhất về tổng thể; điều này thường quan trọng hơn việc tìm truy vấn chậm nhất của một lần gọi riêng lẻ.

View pg_stat_activity cho biết các truy vấn đang chạy và các phiên 'idle in transaction'. Đặt log_min_duration_statement = 500 để ghi log mọi câu lệnh chạy lâu hơn 500 mili giây, và dùng auto_explain để tự động ghi kế hoạch của các câu lệnh chậm đó. Với hệ thống OLTP, tỷ lệ cache hit của bộ nhớ đệm nên trên 99%.

Tình huống minh họa (số liệu không phải đo thực tế): bảng orders có 48 triệu dòng. API 'danh sách đơn hàng của khách' có thời gian phản hồi p95 là 2,8 giây. EXPLAIN (ANALYZE, BUFFERS) cho thấy Seq Scan trên toàn bộ bảng và loại bỏ gần 48 triệu dòng. Nhóm tạo chỉ mục CREATE INDEX CONCURRENTLY ON orders (customer_id, created_at DESC); việc tạo mất khoảng 6 phút và chỉ mục nặng khoảng 1,4GB. Sau đó p95 giảm còn 12 mili giây. Đổi lại, tốc độ INSERT chậm đi khoảng 4%.

Danh sách kiểm tra khi truy vấn chậm: (1) chạy EXPLAIN (ANALYZE, BUFFERS) để xem kế hoạch thực tế; (2) kiểm tra ước tính số dòng có khớp thực tế không, nếu lệch thì chạy ANALYZE; (3) xem điều kiện lọc có dùng hàm hoặc ép kiểu làm mất chỉ mục không; (4) kiểm tra work_mem nếu thấy sắp xếp tràn ra đĩa; (5) kiểm tra dòng chết và lần autovacuum gần nhất; (6) kiểm tra số kết nối và các phiên idle in transaction; (7) chỉ khi các bước trên chưa đủ mới cân nhắc phân vùng hoặc tăng phần cứng.